

# 3-Tier Architecture on AWS

Web Tier • Application Tier • Database Tier

---

VPC Design, Subnetting, Routing, Compute, Database & Security

Prepared from AWS Console walkthrough

# Project Overview

What a 3-tier architecture is and why we build it this way

## The Three Tiers

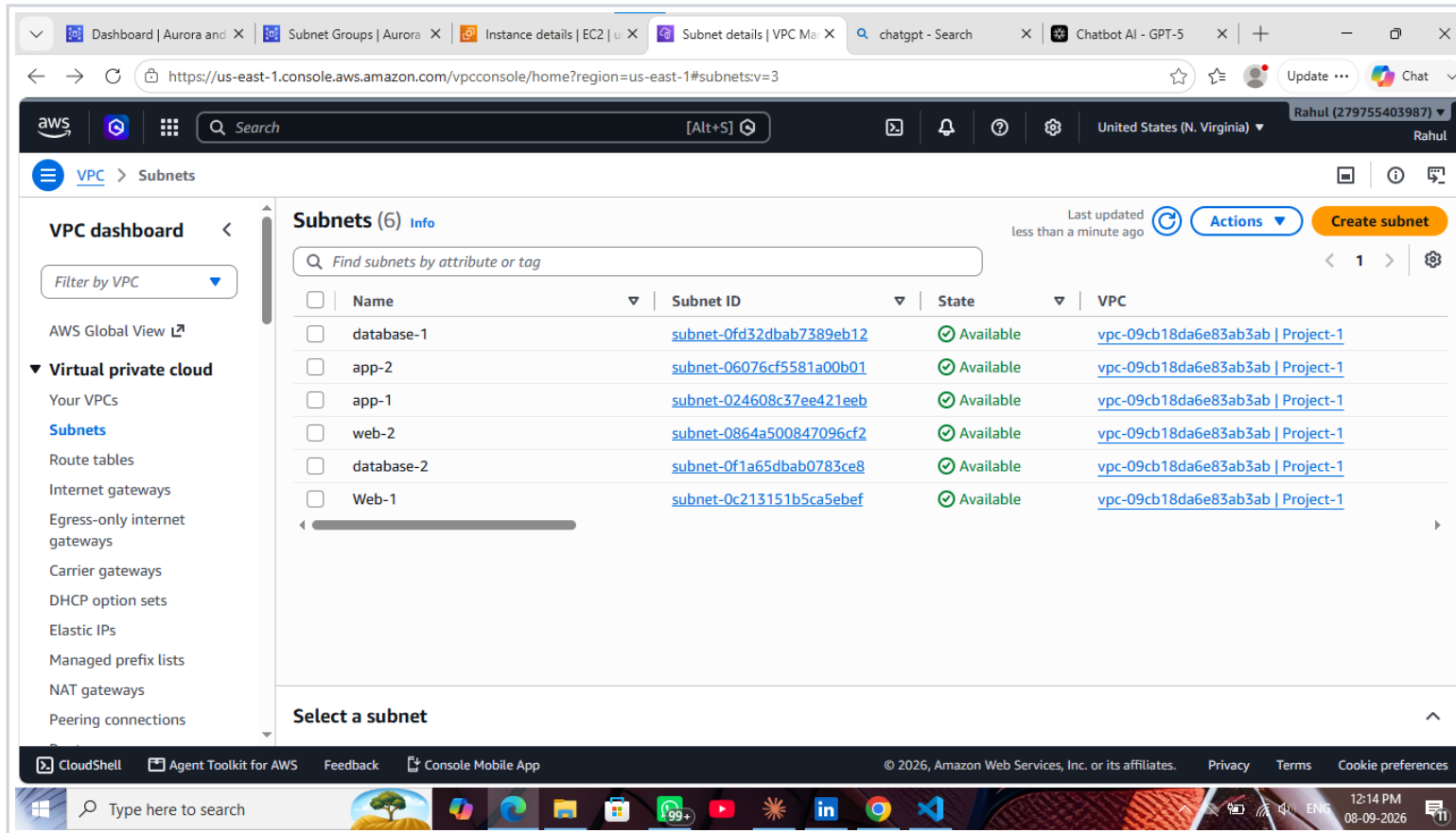
- Web Tier — public-facing subnets that host load balancers / web servers, reachable from the internet
- Application Tier — private subnets running app logic; only reachable from the web tier
- Database Tier — most restricted private subnets, isolated from the internet, only reachable from the app tier

## What this deck walks through

- Custom VPC and subnet layout (public + private across Availability Zones)
- Route tables — public (internet-routed) vs. private (local-only)
- NAT Gateway check for private-subnet outbound access
- Compute layer — EC2 instances across tiers, and Security Groups
- Database layer — Amazon RDS in the isolated database subnets
- Storage — EBS volumes backing the instances

# VPC & Subnet Layout

Six subnets spanning the three tiers across Availability Zones



Subnets view — web, app, and database subnets, each duplicated across AZs

- web-1 / web-2 — public web tier subnets
- app-1 / app-2 — private application tier
- database-1 / database-2 — private, most restricted tier
- Each tier spans 2 subnets for high availability

# Subnet Configuration

Defining CIDR blocks and Availability Zones per subnet

- Subnets are carved out of the VPC's 10.0.0.0/16 CIDR block
- Each subnet is pinned to a specific Availability Zone
- Public subnets get routable, internet-facing CIDR ranges
- Private (app/db) subnets get smaller, isolated ranges

Home | VPC Console x CreateSubnet | VPC Console x +

https://eu-north-1.console.aws.amazon.com/vpcconsole/home?region=eu-north-1#CreateSubnet:VpcId=vpc-0356f21be7ee2b25d

aws [Search] [Alt+S] Europe (Stockholm) Rahul (279755403987) Rahul

VPC > Subnets > Create subnet

Specify the CIDR blocks and Availability Zone for the subnet.

**Subnet 1 of 4**

**Subnet name**  
Create a tag with a key of 'Name' and a value that you specify.  
pubsub-1  
The name can be up to 256 characters long.

**Availability Zone** [Info](#)  
Choose the zone in which your subnet will reside, or let Amazon choose one for you.  
Europe (Stockholm) / eun1-az1 (eu-north-1a)

**IPv4 VPC CIDR block** [Info](#)  
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.  
10.0.0.0/16

**IPv4 subnet CIDR block**  
10.0.1.0/24 256 IPs  
< > ^ v

▼ **Tags - optional**

| Key | Value - optional |
|-----|------------------|
|-----|------------------|

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Type here to search 10:54 AM 25-08-2026

*Creating a public subnet (pubsub-1) — 10.0.1.0/24 in eu-north-1a*

# Routing — Overview

Separate route tables isolate public and private traffic

- Public route table — associated with web-tier subnets
- Private route tables — associated with app & database subnets
- Keeping them separate enforces the tier boundaries

The screenshot shows the AWS VPC console interface. At the top, a green banner indicates a successful update for route table `rtb-07e864bb38ea850ef` (pubrt-1). Below this, the 'Route tables (1/3)' section is displayed. A table lists the route tables:

|                                     | Name    | Route table ID                        | Explicit subnet associ... | Edge associations | Main | VPC                  |
|-------------------------------------|---------|---------------------------------------|---------------------------|-------------------|------|----------------------|
| <input type="checkbox"/>            | -       | <a href="#">rtb-04b257e386afcf88d</a> | -                         | -                 | Yes  | <a href="#">vpc-</a> |
| <input checked="" type="checkbox"/> | -       | <a href="#">rtb-0496f980b25803813</a> | -                         | -                 | Yes  | <a href="#">vpc-</a> |
| <input type="checkbox"/>            | pubrt-1 | <a href="#">rtb-07e864bb38ea850ef</a> | 2 subnets                 | -                 | No   | <a href="#">vpc-</a> |

The route table `rtb-0496f980b25803813` is selected, and its details are shown at the bottom of the console.

*Route tables: a public route table (pubrt-1) plus private route tables*

# Public Route Table

Web tier routes 0.0.0.0/0 to the Internet Gateway

- 0.0.0.0/0 → igw-... gives web subnets direct internet access
- 10.0.0.0/16 → local keeps intra-VPC traffic internal
- Associated with 2 subnets (the public web tier)

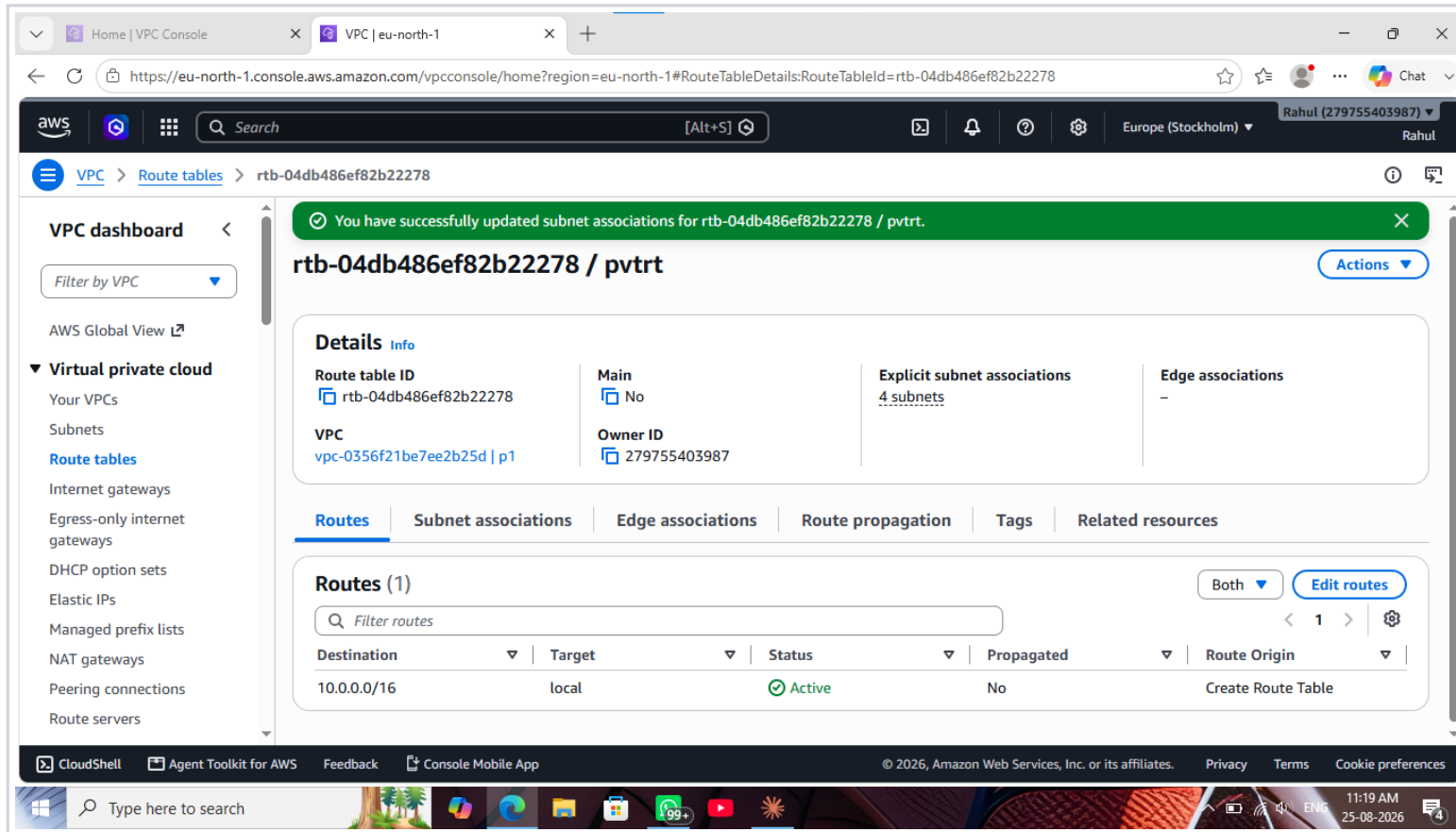
The screenshot displays the AWS VPC console interface for the 'eu-north-1' region. A green notification bar at the top indicates that routes for 'rtb-07e864bb38ea850ef / pubrt-1' were successfully updated. The main content area shows the details for the route table 'rtb-07e864bb38ea850ef / pubrt-1'. The 'Details' tab is active, showing the route table ID, VPC ID, main status, owner ID, and subnet associations. Below this, the 'Routes' tab is selected, displaying a table with two routes. The first route is for destination '0.0.0.0/0' with target 'igw-0e0f5844bac256ff1', status 'Active', and propagated 'No'. The second route is for destination '10.0.0.0/16' with target 'local', status 'Active', and propagated 'No'. The console also shows a sidebar with navigation links for VPC dashboard, subnets, route tables, and other VPC resources.

| Destination | Target                                | Status | Propagated | Route Origin |
|-------------|---------------------------------------|--------|------------|--------------|
| 0.0.0.0/0   | <a href="#">igw-0e0f5844bac256ff1</a> | Active | No         | Create Route |
| 10.0.0.0/16 | local                                 | Active | No         | Create Route |

*pubrt-1 — 0.0.0.0/0 → Internet Gateway, plus local VPC route*

# Private Route Table

App & database tiers have no direct internet route

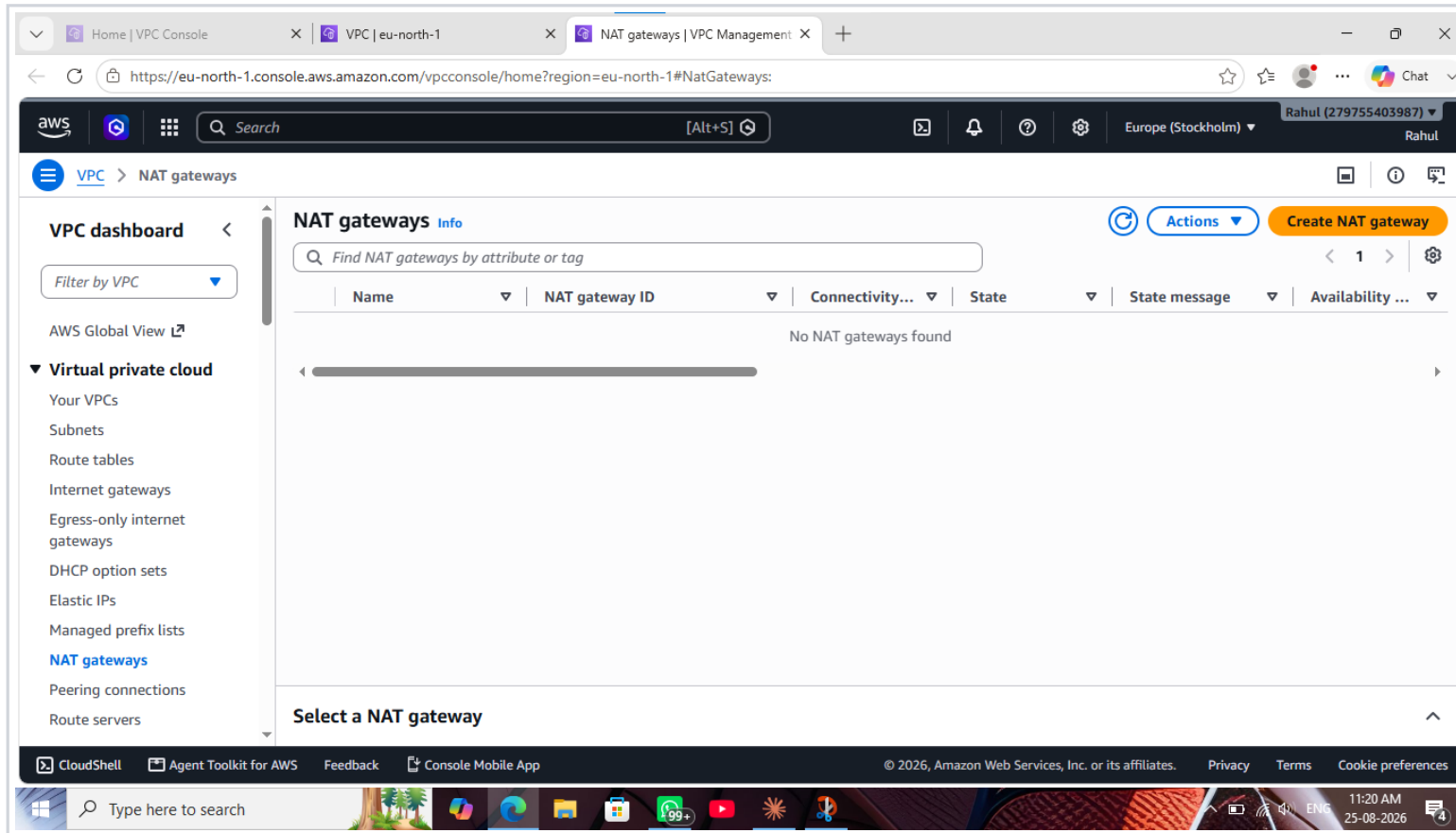


*pvtrt — only a local 10.0.0.0/16 route; associated with 4 private subnets*

- No 0.0.0.0/0 route — not reachable from the internet
- Only the local VPC route (10.0.0.0/16) is present
- Associated with all 4 app + database subnets
- This is what keeps the app & DB tiers isolated

# NAT Gateway

Optional component for private-subnet outbound internet access

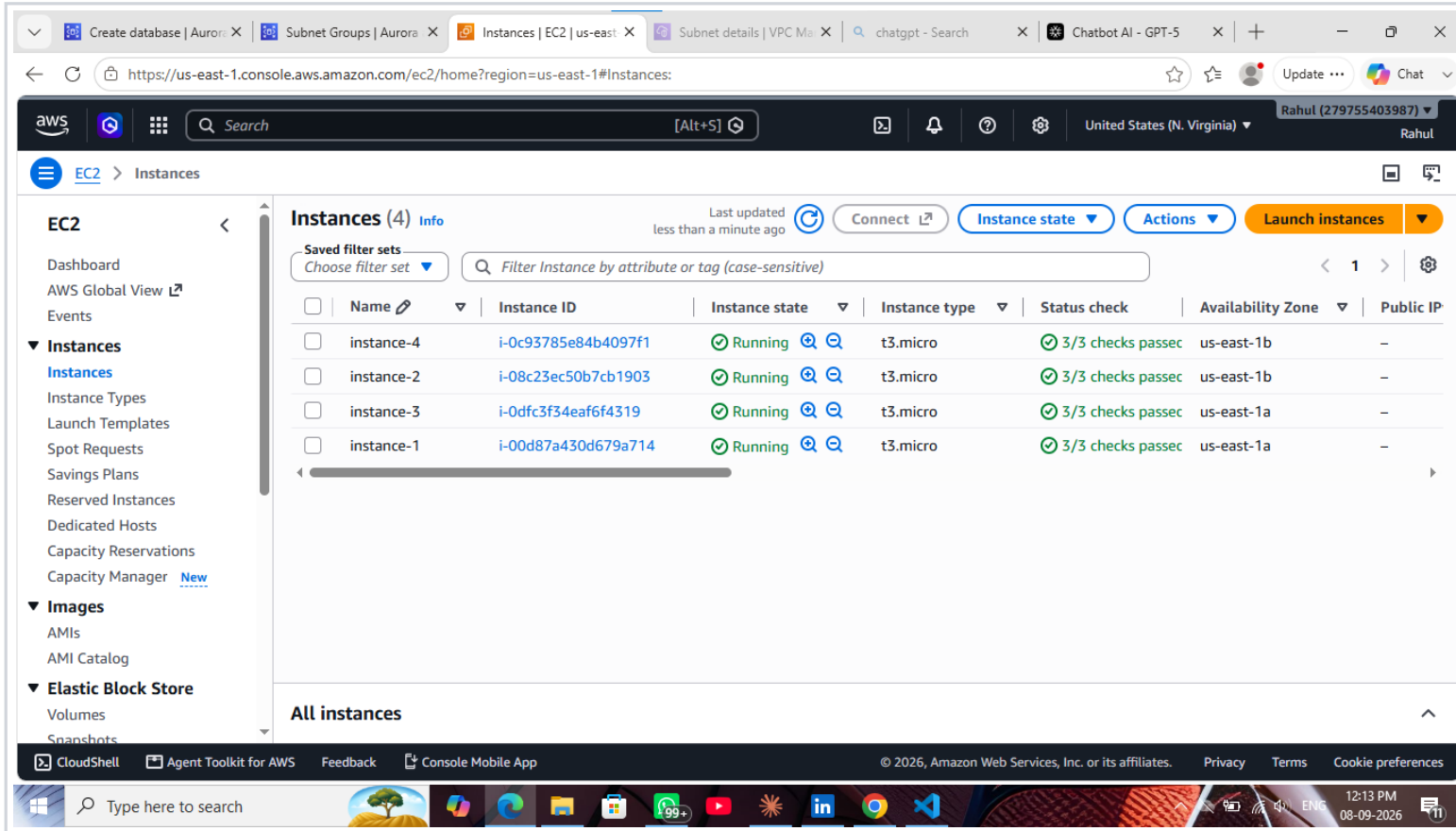


*No NAT gateway provisioned in this build*

- A NAT Gateway would let app/db subnets reach the internet
- outbound-only (e.g. for OS/package updates) — never inbound
- Not provisioned here to keep the environment cost-minimal
- Can be added later in the public subnet if outbound access is needed

# Compute Layer — EC2 Instances

Web and application tiers running on EC2



The screenshot displays the AWS Management Console for the 'Instances' page in the 'us-east-1' region. The left sidebar shows the navigation menu with 'Instances' selected. The main content area shows a table of 4 instances, all in a 'Running' state. The instances are distributed across two Availability Zones: us-east-1a (instances 1 and 3) and us-east-1b (instances 2 and 4). All instances are of type 't3.micro' and have passed all 3 status checks.

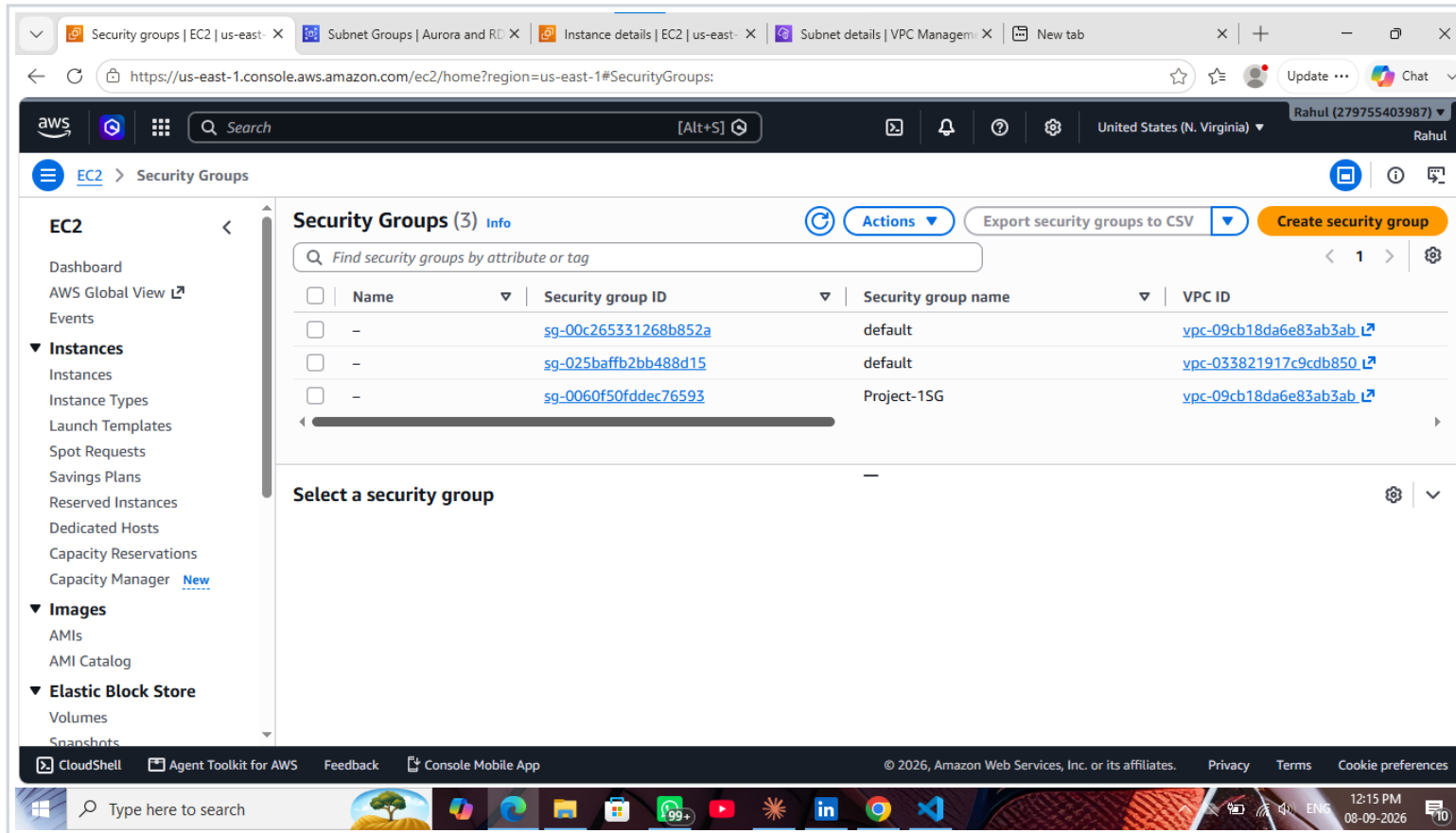
| Name       | Instance ID         | Instance state | Instance type | Status check      | Availability Zone | Public IP |
|------------|---------------------|----------------|---------------|-------------------|-------------------|-----------|
| instance-4 | i-0c93785e84b4097f1 | Running        | t3.micro      | 3/3 checks passed | us-east-1b        | -         |
| instance-2 | i-08c23ec50b7cb1903 | Running        | t3.micro      | 3/3 checks passed | us-east-1b        | -         |
| instance-3 | i-0dfc3f34eaf6f4319 | Running        | t3.micro      | 3/3 checks passed | us-east-1a        | -         |
| instance-1 | i-00d87a430d679a714 | Running        | t3.micro      | 3/3 checks passed | us-east-1a        | -         |

*4 running t3.micro instances spread across two Availability Zones*

- instance-1 & instance-3 in us-east-1a
- instance-2 & instance-4 in us-east-1b
- All instances passing 3/3 status checks
- Spread across AZs for availability

# Security Groups

Instance-level firewalls controlling traffic between tiers

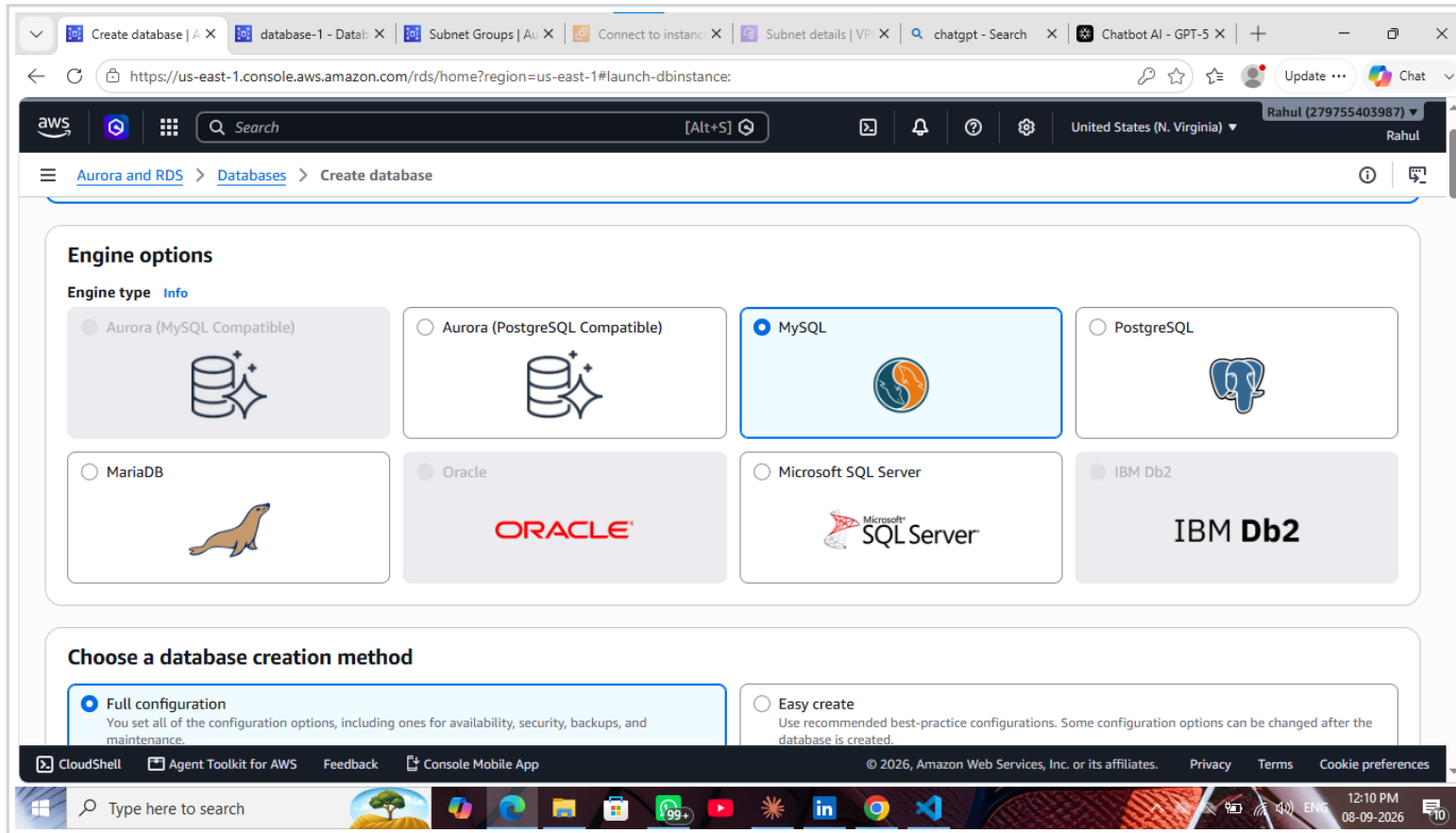


*Project-1SG plus the default security groups for the VPC*

- Project-1SG — custom rules scoped to this project's tiers
- Default groups exist per VPC automatically
- Security groups typically only allow web→app→db traffic
- Complements subnet-level isolation from the route tables

# Database Layer — Engine Selection

Amazon RDS chosen for the database tier

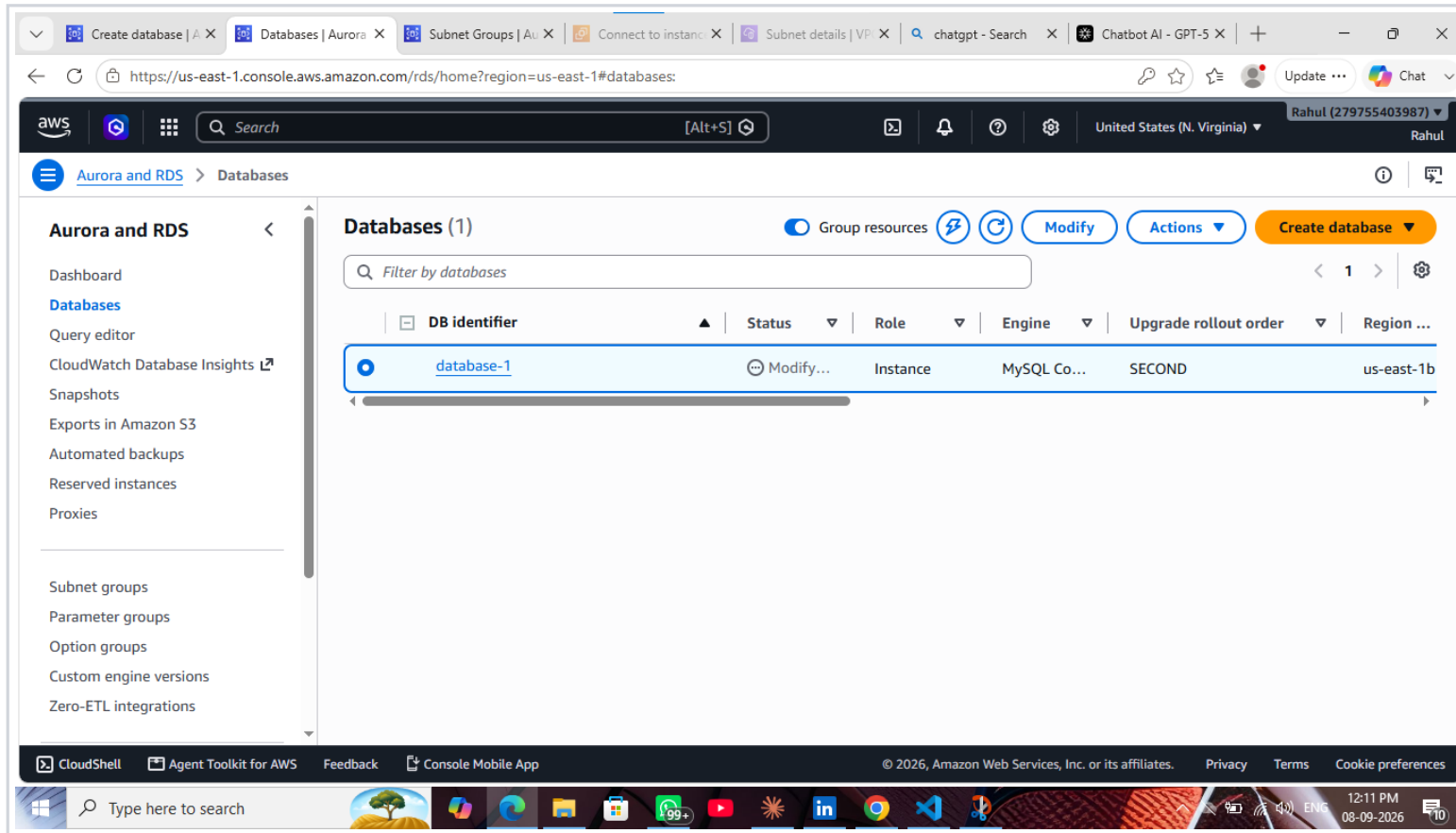


Creating a database — MySQL engine, Full configuration method

- Managed relational database service (RDS)
- Engine: MySQL
- Full configuration — explicit control over availability, security, backups and maintenance settings

# Database Layer — Provisioned

database-1 deployed inside the database-tier subnets



*database-1 — MySQL instance, deployed in us-east-1b*

- Deployed into the private database subnets only
- Not internet-reachable — matches the private route table
- Only the application tier should be permitted to connect

# Storage — EBS Volumes

Block storage backing the EC2 instances

- gp3 volumes, 8 GiB, 3000 IOPS / 125 MB/s throughput
- One root volume per EC2 instance
- No manual snapshots taken yet in the last 24 hours

The screenshot shows the AWS Management Console for the 'us-east-1' region. The left sidebar contains navigation links for EC2, Instances, Images, and Elastic Block Store. The main content area displays 'Volumes (4)' with a table of four gp3 volumes. Each volume is 8 GiB in size, has 3000 IOPS, and a throughput of 125 MB/s. The table also shows the Snapshot ID and Source Volume for each volume. Below the table, there is a section for 'Fault tolerance for all volumes in this Region' and a 'Snapshot summary' indicating that no snapshots have been taken in the last 24 hours.

| Name | Volume ID             | Type | Size  | IOPS | Throughput | Snapshot ID     | Source v |
|------|-----------------------|------|-------|------|------------|-----------------|----------|
|      | vol-0f1b27f50350053ef | gp3  | 8 GiB | 3000 | 125        | snap-0fc7915... | -        |
|      | vol-0077c472f0728e5ba | gp3  | 8 GiB | 3000 | 125        | snap-0fe259e... | -        |
|      | vol-0678a12c3bbca64a7 | gp3  | 8 GiB | 3000 | 125        | snap-0fe259e... | -        |
|      | vol-07c0c32051248fe53 | gp3  | 8 GiB | 3000 | 125        | snap-0fc7915... | -        |

**Snapshot summary**  
Last updated on Tue, Sep 08, 2026, 12:16:20 PM (GMT+05:30)  
Snapshots to back up volumes can be manually created, or created through automation such as [Amazon Data Lifecycle Manager](#), [AWS Backup](#), custom scripts, or third-party tools.  
**Volumes backed up in the last 24 hours / Total # volumes**  
0 / 4  
**Data Lifecycle Manager default policy for EBS Snapshots status**  
No default policy set up | [Create policy](#)

4 gp3 volumes (8 GiB each) — one per EC2 instance

# Summary

End-to-end 3-tier setup on AWS

- VPC (10.0.0.0/16) subdivided into 6 subnets across 2 AZs
- Public route table → Internet Gateway, for the web tier
- Private route table (local only) for app & database tiers
- EC2 instances host the web/app tiers, secured by Security Groups
- Amazon RDS (MySQL) provides the isolated database tier
- EBS volumes provide persistent block storage per instance
- NAT Gateway intentionally omitted to keep the build cost-minimal